```
# CHAPTER 5 Decision Making for Two Samples

# Refer to CHAPTER 4.

# § 5-1 Introduction

# Refer to § 4-1.

# Figure 5-1 Two independent populations.

# § 5-2 Inference on the Means of Two Populations, Variances Known

# Refer to § 4-4.

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(HH, BSDA, ggplot2, purrr, dplyr)

# --- 2. Functions ---
Pwr.z.test <- function(n, delta, sigma, alpha, alt = "two.sided") {
  m <- delta / (sigma / sqrt(n))
  if (alt == "two.sided") {
    return(1 - pnorm(qnorm(1 - alpha/2) - m) + pnorm(qnorm(alpha/2) - m))
  } else if (alt == "greater") {
    return(1 - pnorm(qnorm(1 - alpha) - m))
  } else { # alt == "less"
    return(pnorm(qnorm(alpha) - m))
  }
}

SplSz.z.test <- function(delta, sigma, alpha, pwr, side = 2) {
  ceiling(((qnorm(1 - alpha/side) + qnorm(pwr)) * sigma / delta)^2)
}

SplSz.z.CI <- function(E, sigma, conf.level, side = 2) {
  ceiling((qnorm(1 - (1 - conf.level)/side) * sigma / E)^2)
}
```

```
# § 5-2.1 Hypothesis Testing on the Difference in Means, Variances Known

# Refer to § 4-4.1.

# EXAMPLE 5-1 Paint Drying Time

# Refer to EXAMPLE 4-3.

# --- 3. Parameters ---
(params <- list(Delta0 = 0, sigma = c(8, 8), alpha = 0.05, n = c(10, 10),
            delta = c(-10, 10), pwr.target = 0.90, E = 6,
            conf.target = 0.95)) # δ ≡ Δ − Δ₀ with Δ ≡ μ₁ − μ₂

delta_range <- seq(-20, 20, length.out = 100) # δ ≡ Δ − Δ₀
(n_test_sizes <- c(10, 25, 50, 100)) # n_1 = n_2 = n = 10, 25, 50 and 100

# --- 4. Basic Stats & Data Simulation ---
(xbar <- c(121, 112))
(se <- params$sigma / sqrt(params$n))
(se0 <- sqrt(sum(params$sigma^2 / params$n)))
(sd0 <- sqrt(sum(params$sigma^2)))
(n0 <- (sd0 / se0)^2) # se0 ≡ sd0 / sqrt(n0)
(z0 <- (-diff(xbar) - params$Delta0) / se0)
(Delta <- params$Delta0 + params$delta)

set.seed(123)
Time1_raw <- rnorm(params$n[1], xbar[1], params$sigma[1])
Time2_raw <- rnorm(params$n[2], xbar[2], params$sigma[2])

# Rescale to have sample mean xbar
Time1 <- xbar[1] + (Time1_raw - mean(Time1_raw))
Time2 <- xbar[2] + (Time2_raw - mean(Time2_raw))
```

```
# --- 5. Two-Sided Test Analysis ---
# ==========================================================================
# CASE: TWO-SIDED TEST
# ==========================================================================
cat("\n== TWO-SIDED TEST ANALYSIS ==\n")
(CritVal_2s <- qnorm(c(params$alpha/2, 1 - params$alpha/2)))
(PVal_2s    <- 2 * (1 - pnorm(abs(z0))))
(CI_2s      <- -diff(xbar) + CritVal_2s * se0)
z.test(Time1, Time2, sigma.x = params$sigma[1], sigma.y = params$sigma[2])
cat(sprintf("Test Statistic (z0): %.2f\nP-Value: %.4f\n", z0, PVal_2s))

cat("Confidence Interval:", "[", CI_2s[1], ",", CI_2s[2], "]\n")

# 5.1 Power at δ = −10, 10:
Pwr.z.test(n0, params$delta, sd0, params$alpha)

# 5.2 Power Curves for n1 = n2 = n = 10, 25, 50 and 100
power_2s_df <- expand.grid(delta = delta_range, n = n_test_sizes) %>%
               mutate(power = mapply(Pwr.z.test, n = n, delta = delta,
                                     MoreArgs = list(sigma = sd0,
                                                     alpha = params$alpha)))

# 5.3 Sample Size Requirements (n_1 = n_2 = n)
# a. For Hypothesis Testing
(n_req_2s <- SplSz.z.test(params$delta, sd0, params$alpha,
                          params$pwr.target))
cat(sprintf("Required n1 = n2 = n for Power ≥ %.2f at Δ = %s: %s\n",
      params$pwr.target, paste(Delta, collapse = ", "),
      paste(n_req_2s, collapse = ", ")))
```

```
cat("\nPower at Δ =", Delta[1], "when n1 = n2 = n =", n_req_2s[1] - 1,
  ",", n_req_2s[1], ":",
  Pwr.z.test(c(n_req_2s[1] - 1, n_req_2s[1]), params$delta[1], sd0,
    params$alpha),
  "\n", "\nPower at Δ = ", Delta[2], "when n1 = n2 = n =",
  n_req_2s[2] - 1, ",", n_req_2s[2], ":",
  Pwr.z.test(c(n_req_2s[2] - 1, n_req_2s[2]), params$delta[2], sd0,
    params$alpha),
  "\n")

# b. For Confidence Interval
(SplSz_CI_2s <- SplSz.z.CI(params$E, sd0, params$conf.target))
cat(sprintf("Required n1 = n2 = n for Error Margin = %s and Confidence ≥
%.2f: %d\n",
      params$E, params$conf.target, SplSz_CI_2s))

cat("\nNote: {|X1¯ − X2¯ − (μ1 − μ2)| ≤ E} =
      {μ1 − μ2 ε [X1¯ − X2¯ − E, X1¯ − X2¯ + E]} and
      CI = [x1¯ − x2¯ − E, x1¯ − x2¯ + E] = [", -diff(xbar) - params$E,
",",
  -diff(xbar) + params$E, "].
n1 = n2 = n =", SplSz_CI_2s - 1, ":", 100 * params$conf.target, "% CI = [",
  -diff(xbar) + CritVal_2s[1] * sd0 / sqrt(SplSz_CI_2s - 1), ",",
  -diff(xbar) + CritVal_2s[2] * sd0 / sqrt(SplSz_CI_2s - 1), "]
n1 = n2 = n =", SplSz_CI_2s, ":", 100 * params$conf.target, "% CI = [",
  -diff(xbar) + CritVal_2s[1] * sd0 / sqrt(SplSz_CI_2s), ",",
  -diff(xbar) + CritVal_2s[2] * sd0 / sqrt(SplSz_CI_2s), "]\n")

# 5.4 Visualizations
# a. Hypothesis Testing (Current n1 = n2 = n = 10)
# Showing the Effect of δ (≡ Δ − Δ₀ with Δ ≡ μ₁ − μ₂) on Power
normal.and.t.dist(params$Delta0, Delta[1], -diff(xbar), sd0, n0, NA,
  params$alpha/2, params$alpha/2, T, T, T)
normal.and.t.dist(params$Delta0, Delta[2], -diff(xbar), sd0, n0, NA,
  params$alpha/2, params$alpha/2, T, T, T)
```

```
# b. Confidence Interval (Current n1 = n2 = n = 10)
normal.and.t.dist(NA, NA, -diff(xbar), sd0, n0, NA, params$alpha/2,
  params$alpha/2, F, T, T, T, "c")

# c. Power Curves for n1 = n2 = n = 10, 25, 50 and 100
ggplot(power_2s_df, aes(x = delta, y = power, color = as.factor(n))) +
  geom_line(linewidth = 1) +
  geom_hline(yintercept = params$pwr.target, linetype = "dashed",
    color = "red") +
  labs(title = "Power Curves for Two-Sample Z-Test (Known Variances)",
    subtitle =
    expression(H[0]*": "*Delta*" = "*Delta[0]*" vs. "*H[1]*": "*Delta*" ≠
"*Delta[0]),
    x = expression(delta == Delta - Delta[0]), y = "Power",
    color = "Sample Size (n)")

# --- 6. One-Sided (Greater) Test Analysis ---
# ==========================================================================
# CASE: ONE-SIDED (GREATER) TEST
# ==========================================================================
cat("\n=== ONE-SIDED (GREATER) TEST ANALYSIS ===\n")
(CritVal_gr <- qnorm(c(params$alpha, 1)))
(PVal_gr    <- 1 - pnorm(z0))
(CI_gr      <- -diff(xbar) + CritVal_gr * se0)
z.test(Time1, Time2, alternative = "greater", sigma.x = params$sigma[1],
  sigma.y = params$sigma[2])
cat(sprintf("Test Statistic (z0): %.2f\nP-Value: %.4f\n", z0, PVal_gr))

cat("Confidence Interval:", "[", CI_gr[1], ", ∞)\n")

# 6.1 Power at δ = 10:
Pwr.z.test(n0, params$delta[2], sd0, params$alpha, "greater")
```

```
# 6.2 Power Curves for n1 = n2 = n = 10, 25, 50 and 100
power_gr_df <- expand.grid(delta = delta_range, n = n_test_sizes) %>%
               mutate(power = mapply(Pwr.z.test, n = n, delta = delta,
                                     MoreArgs = list(sigma = sd0,
                                                     alpha = params$alpha,
                                                     alt = "greater")))

# 6.3 Sample Size Requirements (n1 = n2 = n)
# a. For Hypothesis Testing
(n_req_gr <- SplSz.z.test(params$delta[2], sd0, params$alpha,
                          params$pwr.target, 1))
cat(sprintf("Required n1 = n2 = n for Power ≥ %.2f at Δ = %s: %s\n",
      params$pwr.target, Delta[2], n_req_gr))

cat("\nPower at Δ = ", Delta[2], "when n1 = n2 = n =", n_req_2s[2] - 1,
  ",", n_req_2s[2], ":",
  Pwr.z.test(c(n_req_2s[2] - 1, n_req_2s[2]), params$delta[2], sd0,
    params$alpha),
  "\n")

# b. For Confidence Interval
(SplSz_CI_gr <- SplSz.z.CI(params$E, sd0, params$conf.target, 1))
cat(sprintf("Required n1 = n2 = n for Error Margin = %s and Confidence ≥
%.2f: %d\n",
      params$E, params$conf.target, SplSz_CI_gr))

cat("\nNote: {X1¯ − X2¯ − (μ1 − μ2) ≤ E} =
      {μ1 − μ2 ε [X1¯ − X2¯ − E, ∞)} and
      CI = [x1¯ − x2¯ − E, ∞) = [", -diff(xbar) - params$E, ",", "∞).
n1 = n2 = n =", SplSz_CI_gr - 1, ":", 100 * params$conf.target, "% CI = [",
  -diff(xbar) + CritVal_gr[1] * sd0 / sqrt(SplSz_CI_gr - 1), ", ∞)\
n1 = n2 = n =", SplSz_CI_gr, ":", 100 * params$conf.target, "% CI = [",
  -diff(xbar) + CritVal_gr[1] * sd0 / sqrt(SplSz_CI_gr), ", ∞)\n")
```

```
# 6.4 Visualizations
# a. Hypothesis Testing (Current n1 = n2 = n = 10)
# Showing the Effect of δ (≡ Δ − Δ₀ with Δ ≡ μ₁ − μ₂) on Power
normal.and.t.dist(params$Delta0, Delta[2], -diff(xbar), sd0, n0, NA,
  params$alpha, params$alpha, T, T)

# b. Confidence Interval (Current n1 = n2 = n = 10)
normal.and.t.dist(NA, NA, -diff(xbar), sd0, n0, NA, params$alpha,
  params$alpha, F, F, T, F, "c")

# c. Power Curves for n1 = n2 = n = 10, 25, 50 and 100
ggplot(power_gr_df, aes(x = delta, y = power, color = as.factor(n))) +
  geom_line(linewidth = 1) +
  geom_hline(yintercept = params$pwr.target, linetype = "dashed",
    color = "red") +
  labs(title = "Power Curves for Two-Sample Z-Test (Known Variances)",
    subtitle =
    expression(H[0]*": "*Delta*" ≤ "*Delta[0]*" vs. "*H[1]*": "*Delta*" >
"*Delta[0]),
    x = expression(delta == Delta - Delta[0]), y = "Power",
    color = "Sample Size (n)")

# --- 7. One-Sided (Less) Test Analysis ---
# ========================================================================
# CASE: ONE-SIDED (LESS) TEST
# ========================================================================
cat("\n=== ONE-SIDED (LESS) TEST ANALYSIS ===\n")
(CritVal_le <- qnorm(c(0, 1 - params$alpha)))
(PVal_le    <- pnorm(z0))
(CI_le      <- -diff(xbar) + CritVal_le * se0)
z.test(Time1, Time2, alternative = "less", sigma.x = params$sigma[1],
  sigma.y = params$sigma[2])
cat(sprintf("Test Statistic (z0): %.2f\nP-Value: %.4f\n", z0, PVal_le))

cat("Confidence Interval:", "(−∞,", CI_le[2], "]\n")
```

```
# 7.1 Power at δ = −10:
Pwr.z.test(n0, params$delta[1], sd0, params$alpha, "less")

# 7.2 Power Curves for n1 = n2 = n = 10, 25, 50 and 100
power_le_df <- expand.grid(delta = delta_range, n = n_test_sizes) %>%
               mutate(power = mapply(Pwr.z.test, n = n, delta = delta,
                                     MoreArgs = list(sigma = sd0,
                                                     alpha = params$alpha,
                                                     alt = "less")))

# 7.3 Sample Size Requirements (n1 = n2 = n)
# a. For Hypothesis Testing
(n_req_le <- SplSz.z.test(params$delta[1], sd0, params$alpha,
                          params$pwr.target, 1))
cat(sprintf("Required n1 = n2 = n for Power ≥ %.2f at Δ = %s: %s\n",
      params$pwr.target, Delta[1], n_req_le))

cat("\nPower at Δ =", Delta[1], "when n1 = n2 = n =", n_req_le - 1, ",",
  n_req_le, ":",
  Pwr.z.test(c(n_req_le - 1, n_req_le), params$delta[1], sd0, params$alpha,
    "less"),
  "\n")

# b. For Confidence Interval
(SplSz_CI_le <- SplSz.z.CI(params$E, sd0, params$conf.target, 1))
cat(sprintf("Required n1 = n2 = n for Error Margin = %s and Confidence ≥
%.2f: %d\n",
      params$E, params$conf.target, SplSz_CI_le))

cat("\nNote: {X1¯ − X2¯ − (μ1 − μ2) ≥ −E} =
      {μ1 − μ2 ε (−∞, X1¯ − X2¯ + E]} and
      CI = (−∞, x1¯ − x2¯ + E] = (−∞,", -diff(xbar) + params$E, "].
n1 = n2 = n =", SplSz_CI_le - 1, ":", 100 * params$conf.target,
  "% CI = (−∞,",
  -diff(xbar) + CritVal_le[2] * sd0 / sqrt(SplSz_CI_le - 1), "]
n1 = n2 = n =", SplSz_CI_le, ":", 100 * params$conf.target, "% CI = (−∞,",
  -diff(xbar) + CritVal_le[2] * sd0 / sqrt(SplSz_CI_le), "]\n")
```

```
# 7.4 Visualizations
# a. Hypothesis Testing (Current n1 = n2 = n = 10)
# Showing the Effect of δ (≡ Δ − Δ₀ with Δ ≡ μ₁ − μ₂) on Power
normal.and.t.dist(params$Delta0, Delta[1], -diff(xbar), sd0, n0, NA,
  params$alpha, params$alpha, T, T, T, F)

# b. Confidence Interval (Current n1 = n2 = n = 10)
normal.and.t.dist(NA, NA, -diff(xbar), sd0, n0, NA, params$alpha,
  params$alpha, F, T, F, T, "c")

# c. Power Curves for n1 = n2 = n = 10, 25, 50 and 100
ggplot(power_le_df, aes(x = delta, y = power, color = as.factor(n))) +
  geom_line(linewidth = 1) +
  geom_hline(yintercept = params$pwr.target, linetype = "dashed",
    color = "red") +
  labs(title = "Power Curves for Two-Sample Z-Test (Known Variances)",
    subtitle =
    expression(H[0]*": "*Delta*" ≥ "*Delta[0]*" vs. "*H[1]*": "*Delta*" <
"*Delta[0]),
    x = expression(delta == Delta - Delta[0]), y = "Power",
    color = "Sample Size (n)")

# § 5-2.2 Type II Error and Choice of Sample Size

# Refer to § 4-4.2.

# EXAMPLE 5-2 Paint Drying Time

# Refer to EXAMPLES 4-4 and 5-1.

# § 5-2.3 Confidence Interval on the Difference in Means, Variances Known

# Refer to § 4-4.5.

# EXAMPLE 5-3 Aircraft Spars

# Table 5-1 Tensile Strength Test Result for Aluminum Spars
```

```
# Refer to EXAMPLE 5-1.

# --- 3*. Parameters ---
(params <- list(Delta0 = 0, sigma = c(1.0, 1.5), alpha = 0.1,
            n = c(10, 12), delta = c(-1, 1), pwr.target = 0.9, E = 0.6,
            conf.target = 0.9)) # δ ≡ Δ − Δ₀ with Δ ≡ μ₁ − μ₂

delta_range <- seq(-3, 3, length.out = 100) # δ ≡ Δ − Δ₀
(n_test_sizes <- c(10, 25, 50, 100)) # n_1 = n_2 = n = 10, 25, 50 and 100

# --- 4*. Basic Stats & Data Simulation ---
(xbar <- c(87.6, 74.5))
(se <- params$sigma / sqrt(params$n))
(se0 <- sqrt(sum(params$sigma^2 / params$n)))
(sd0 <- sqrt(sum(params$sigma^2)))
(n0 <- (sd0 / se0)^2) # se0 ≡ sd0 / sqrt(n0)
(z0 <- (-diff(xbar) - params$Delta0) / se0)
(Delta <- params$Delta0 + params$delta)

set.seed(123)
TS1_raw <- rnorm(params$n[1], xbar[1], params$sigma[1])
TS2_raw <- rnorm(params$n[2], xbar[2], params$sigma[2])

# Rescale to have sample mean xbar
TS1 <- xbar[1] + (TS1_raw - mean(TS1_raw))
TS2 <- xbar[2] + (TS2_raw - mean(TS2_raw))
```

```
# --- 5*. Two-Sided Test Analysis ---
# ==========================================================================
# CASE: TWO-SIDED TEST
# ==========================================================================
cat("\n== TWO-SIDED TEST ANALYSIS ==\n")
(CritVal_2s <- qnorm(c(params$alpha/2, 1 - params$alpha/2)))
(PVal_2s    <- 2 * (1 - pnorm(abs(z0))))
(CI_2s      <- -diff(xbar) + CritVal_2s * se0)
z.test(TS1, TS2, sigma.x = params$sigma[1], sigma.y = params$sigma[2],
       conf.level = 0.9)
cat(sprintf("Test Statistic (z0): %.2f\nP-Value: %.4f\n", z0, PVal_2s))

cat("Confidence Interval:", "[", CI_2s[1], ",", CI_2s[2], "]\n")

# 5*.1 Power at δ = −1, 1:
Pwr.z.test(n0, params$delta, sd0, params$alpha)

# 5*.2 Power Curves for n_1 = n_2 = n = 10, 25, 50 and 100
power_2s_df <- expand.grid(delta = delta_range, n = n_test_sizes) %>%
               mutate(power = mapply(Pwr.z.test, n = n, delta = delta,
                                     MoreArgs = list(sigma = sd0,
                                                     alpha = params$alpha)))

# 5*.3 Sample Size Requirements (n_1 = n_2 = n)
# a. For Hypothesis Testing
(n_req_2s <- SplSz.z.test(params$delta, sd0, params$alpha,
                          params$pwr.target))
cat(sprintf("Required n1 = n2 = n for Power ≥ %.2f at Δ = %s: %s\n",
      params$pwr.target, paste(Delta, collapse = ", "),
      paste(n_req_2s, collapse = ", ")))
```

```
cat("\nPower at Δ =", Delta[1], "when n1 = n2 = n =", n_req_2s[1] - 1,
  ",", n_req_2s[1], ":",
  Pwr.z.test(c(n_req_2s[1] - 1, n_req_2s[1]), params$delta[1], sd0,
    params$alpha),
  "\n", "\nPower at Δ = ", Delta[2], "when n1 = n2 = n =",
  n_req_2s[2] - 1, ",", n_req_2s[2], ":",
  Pwr.z.test(c(n_req_2s[2] - 1, n_req_2s[2]), params$delta[2], sd0,
    params$alpha),
  "\n")

# b. For Confidence Interval
(SplSz_CI_2s <- SplSz.z.CI(params$E, sd0, params$conf.target))
cat(sprintf("Required n1 = n2 = n for Error Margin = %s and Confidence ≥
%.2f: %d\n",
      params$E, params$conf.target, SplSz_CI_2s))

cat("\nNote: {|X1¯ − X2¯ − (μ1 − μ2)| ≤ E} =
      {μ1 − μ2 ε [X1¯ − X2¯ − E, X1¯ − X2¯ + E]} and
      CI = [x1¯ − x2¯ − E, x1¯ − x2¯ + E] = [", -diff(xbar) - params$E,
",",
  -diff(xbar) + params$E, "].
n1 = n2 = n =", SplSz_CI_2s - 1, ":", 100 * params$conf.target, "% CI = [",
  -diff(xbar) + CritVal_2s[1] * sd0 / sqrt(SplSz_CI_2s - 1), ",",
  -diff(xbar) + CritVal_2s[2] * sd0 / sqrt(SplSz_CI_2s - 1), "]
n1 = n2 = n =", SplSz_CI_2s, ":", 100 * params$conf.target, "% CI = [",
  -diff(xbar) + CritVal_2s[1] * sd0 / sqrt(SplSz_CI_2s), ",",
  -diff(xbar) + CritVal_2s[2] * sd0 / sqrt(SplSz_CI_2s), "]\n")

# 5*.4 Visualizations
# a. Hypothesis Testing (Current n1 = 10 and n2 = 12)
# Showing the Effect of δ (≡ Δ − Δ₀ with Δ ≡ μ₁ − μ₂) on Power
normal.and.t.dist(params$Delta0, Delta[1], -diff(xbar), sd0, n0, NA,
  params$alpha/2, params$alpha/2, T, T, T)
normal.and.t.dist(params$Delta0, Delta[2], -diff(xbar), sd0, n0, NA,
  params$alpha/2, params$alpha/2, T, T, T)
```

```
# b. Confidence Interval (Current n1 = 10 and n2 = 12)
normal.and.t.dist(NA, NA, -diff(xbar), sd0, n0, NA, params$alpha/2,
  params$alpha/2, F, T, T, T, "c")

# c. Power Curves for n1 = n2 = n = 10, 25, 50 and 100
ggplot(power_2s_df, aes(x = delta, y = power, color = as.factor(n))) +
  geom_line(linewidth = 1) +
  geom_hline(yintercept = params$pwr.target, linetype = "dashed",
    color = "red") +
  labs(title = "Power Curves for Two-Sample Z-Test (Known Variances)",
    subtitle =
    expression(H[0]*": "*Delta*" = "*Delta[0]*" vs. "*H[1]*": "*Delta*" ≠
"*Delta[0]),
    x = expression(delta == Delta - Delta[0]), y = "Power",
    color = "Sample Size (n)")

# --- 6*. One-Sided (Greater) Test Analysis ---
# ==========================================================================
# CASE: ONE-SIDED (GREATER) TEST
# ==========================================================================
cat("\n=== ONE-SIDED (GREATER) TEST ANALYSIS ===\n")
(CritVal_gr <- qnorm(c(params$alpha, 1)))
(PVal_gr    <- 1 - pnorm(z0))
(CI_gr      <- -diff(xbar) + CritVal_gr * se0)
z.test(TS1, TS2, alternative = "greater", sigma.x = params$sigma[1],
  sigma.y = params$sigma[2], conf.level = 0.9)
cat(sprintf("Test Statistic (z0): %.2f\nP-Value: %.4f\n", z0, PVal_gr))

cat("Confidence Interval:", "[", CI_gr[1], ", ∞)\n")

# 6*.1 Power at δ = 1:
Pwr.z.test(n0, params$delta[2], sd0, params$alpha, "greater")
```

```
# 6*.2 Power Curves for n1 = n2 = n = 10, 25, 50 and 100
power_gr_df <- expand.grid(delta = delta_range, n = n_test_sizes) %>%
               mutate(power = mapply(Pwr.z.test, n = n, delta = delta,
                                     MoreArgs = list(sigma = sd0,
                                                     alpha = params$alpha,
                                                     alt = "greater")))

# 6*.3 Sample Size Requirements (n1 = n2 = n)
# a. For Hypothesis Testing
(n_req_gr <- SplSz.z.test(params$delta[2], sd0, params$alpha,
                          params$pwr.target, 1))
cat(sprintf("Required n1 = n2 = n for Power ≥ %.2f at Δ = %s: %s\n",
      params$pwr.target, Delta[2], n_req_gr))

# b. For Confidence Interval
(SplSz_CI_gr <- SplSz.z.CI(params$E, sd0, params$conf.target, 1))
cat(sprintf("Required n1 = n2 = n for Error Margin = %s and Confidence ≥
%.2f: %d\n",
      params$E, params$conf.target, SplSz_CI_gr))

cat("\nNote: {X1¯ − X2¯ − (μ1 − μ2) ≤ E} =
      {μ1 − μ2 ε [X1¯ − X2¯ − E, ∞)} and
      CI = [x1¯ − x2¯ − E, ∞) = [", -diff(xbar) - params$E, ",", "∞).
n1 = n2 = n =", SplSz_CI_gr - 1, ":", 100 * params$conf.target, "% CI = [",
  -diff(xbar) + CritVal_gr[1] * sd0 / sqrt(SplSz_CI_gr - 1), ", ∞)\
n1 = n2 = n =", SplSz_CI_gr, ":", 100 * params$conf.target, "% CI = [",
  -diff(xbar) + CritVal_gr[1] * sd0 / sqrt(SplSz_CI_gr), ", ∞)\n")

# 6*.4 Visualizations
# a. Hypothesis Testing (Current n1 = 10 and n2 = 12)
# Showing the Effect of δ (≡ Δ − Δ₀ with Δ ≡ μ₁ − μ₂) on Power
normal.and.t.dist(params$Delta0, Delta[2], -diff(xbar), sd0, n0, NA,
  params$alpha, params$alpha, T, T)

# b. Confidence Interval (Current n1 = 10 and n2 = 12)
normal.and.t.dist(NA, NA, -diff(xbar), sd0, n0, NA, params$alpha,
  params$alpha, F, F, T, F, "c")
```

```
# c. Power Curves for n1 = n2 = n = 10, 25, 50 and 100
ggplot(power_gr_df, aes(x = delta, y = power, color = as.factor(n))) +
  geom_line(linewidth = 1) +
  geom_hline(yintercept = params$pwr.target, linetype = "dashed",
    color = "red") +
  labs(title = "Power Curves for Two-Sample Z-Test (Known Variances)",
    subtitle =
    expression(H[0]*": "*Delta*" ≤ "*Delta[0]*" vs. "*H[1]*": "*Delta*" >
"*Delta[0]),
    x = expression(delta == Delta - Delta[0]), y = "Power",
    color = "Sample Size (n)")

# --- 7*. One-Sided (Less) Test Analysis ---
# ==========================================================================
# CASE: ONE-SIDED (LESS) TEST
# ==========================================================================
cat("\n=== ONE-SIDED (LESS) TEST ANALYSIS ===\n")
(CritVal_le <- qnorm(c(0, 1 - params$alpha)))
(PVal_le    <- pnorm(z0))
(CI_le      <- -diff(xbar) + CritVal_le * se0)
z.test(TS1, TS2, alternative = "less", sigma.x = params$sigma[1],
  sigma.y = params$sigma[2], conf.level = 0.9)
cat(sprintf("Test Statistic (z0): %.2f\nP-Value: %.4f\n", z0, PVal_le))

cat("Confidence Interval:", "(−∞,", CI_le[2], "]\n")

# 7*.1 Power at δ = −1:
Pwr.z.test(n0, params$delta[1], sd0, params$alpha, "less")

# 7*.2 Power Curves for n1 = n2 = n = 10, 25, 50 and 100
power_le_df <- expand.grid(delta = delta_range, n = n_test_sizes) %>%
               mutate(power = mapply(Pwr.z.test, n = n, delta = delta,
                                     MoreArgs = list(sigma = sd0,
                                                     alpha = params$alpha,
                                                     alt = "less")))
```

```
# 7*.3 Sample Size Requirements (n1 = n2 = n)
# a. For Hypothesis Testing
(n_req_le <- SplSz.z.test(params$delta[1], sd0, params$alpha,
                          params$pwr.target, 1))
cat(sprintf("Required n1 = n2 = n for Power ≥ %.2f at Δ = %s: %s\n",
      params$pwr.target, Delta[1], n_req_le))

cat("\nPower at Δ =", Delta[1], "when n1 = n2 = n =", n_req_le - 1, ",",
  n_req_le, ":",
  Pwr.z.test(c(n_req_le - 1, n_req_le), params$delta[1], sd0, params$alpha,
    "less"),
  "\n")

# b. For Confidence Interval
(SplSz_CI_le <- SplSz.z.CI(params$E, sd0, params$conf.target, 1))
cat(sprintf("Required n1 = n2 = n for Error Margin = %s and Confidence ≥
%.2f: %d\n",
      params$E, params$conf.target, SplSz_CI_le))

cat("\nNote: {X1¯ − X2¯ − (μ1 − μ2) ≥ −E} =
      {μ1 − μ2 ε (−∞, X1¯ − X2¯ + E]} and
      CI = (−∞, x1¯ − x2¯ + E] = (−∞,", -diff(xbar) + params$E, "].
n1 = n2 = n =", SplSz_CI_le - 1, ":", 100 * params$conf.target,
  "% CI = (−∞,",
  -diff(xbar) + CritVal_le[2] * sd0 / sqrt(SplSz_CI_le - 1), "]
n1 = n2 = n =", SplSz_CI_le, ":", 100 * params$conf.target, "% CI = (−∞,",
  -diff(xbar) + CritVal_le[2] * sd0 / sqrt(SplSz_CI_le), "]\n")

# 7*.4 Visualizations
# a. Hypothesis Testing (Current n1 = 10 and n2 = 12)
# Showing the Effect of δ (≡ Δ − Δ₀ with Δ ≡ μ₁ − μ₂) on Power
normal.and.t.dist(params$Delta0, Delta[1], -diff(xbar), sd0, n0, NA,
  params$alpha, params$alpha, T, T, T, F)

# b. Confidence Interval (Current n1 = 10 and n2 = 12)
normal.and.t.dist(NA, NA, -diff(xbar), sd0, n0, NA, params$alpha,
  params$alpha, F, T, F, T, "c")
```

```
# c. Power Curves for n1 = n2 = n = 10, 25, 50 and 100
ggplot(power_le_df, aes(x = delta, y = power, color = as.factor(n))) +
  geom_line(linewidth = 1) +
  geom_hline(yintercept = params$pwr.target, linetype = "dashed",
    color = "red") +
  labs(title = "Power Curves for Two-Sample Z-Test (Known Variances)",
    subtitle =
    expression(H[0]*": "*Delta*" ≥ "*Delta[0]*" vs. "*H[1]*": "*Delta*" <
"*Delta[0]),
    x = expression(delta == Delta - Delta[0]), y = "Power",
    color = "Sample Size (n)")

# EXERCISES FOR § 5-2
```

§ 5-3 Inference on the Means of Two Populations, Variances Unknown

Refer to § 4-5.

```
# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(car, HH)

# --- 2. Function ---
Pwr.t.test <- function (n, delta, sigma, alpha, alt = "two.sided") {
  df  <- n - 1
  ncp <- delta / (sigma / sqrt(n))
  if (alt == "two.sided") {
    return(1 - pt(qt(1 - alpha/2, df), df, ncp) +
           pt(qt(alpha/2, df), df, ncp))
  } else if (alt == "greater") {
    return(1 - pt(qt(1 - alpha, df), df, ncp))
  } else { # alt == "less"
    return(pt(qt(alpha, df), df, ncp))
  }
}
```

§ 5-3.1 Hypothesis Testing on the Difference in Means

Case 1: $\sigma_1^2 = \sigma_2^2 = \sigma^2$

EXAMPLE 5-4 Chemical Process Yield

Table 5-2 Catalyst Yield Data (Percent) Example 5-4

Refer to EXAMPLE 4-7.

```
# --- 3. Data & Parameters ---
(Cat1 <- c(91.50, 94.18, 92.18, 95.39, 91.79, 89.07, 94.72, 89.21))
(Cat2 <- c(89.19, 90.95, 90.46, 93.21, 97.19, 97.04, 91.07, 92.75))
(params <- list(Delta0 = 0, alpha = 0.05,
            n = c(length(Cat1), length(Cat2)), delta = c(-4, 4),
            pwr.target = 0.85)) # δ ≡ Δ − Δ₀ with Δ ≡ μ₁ − μ₂

# --- 4. Basic Stats, Normality Test, Box Plots & Variance Test ---
summary(Cat1)
summary(Cat2)
(xbar <- c(mean(Cat1), mean(Cat2)))
(s <- c(sd(Cat1), sd(Cat2)))
(df <- params$n - 1)
(df_Case1 <- sum(df))
(s_p <- sqrt(sum(df * s^2) / df_Case1))
(se_Case1 <- s_p * sqrt(sum(1 / params$n)))
(t0 <- (-diff(xbar) - params$Delta0) / se_Case1)
(n_Case1 <- df_Case1 + 1)
(s_Case1 <- sqrt(n_Case1) * se_Case1)
(Delta <- params$Delta0 + params$delta)

# Normality Test
shapiro.test(Cat1)
shapiro.test(Cat2)
Yield <- c(Cat1, Cat2)
Type <- c(rep(1, params$n[1]), rep(2, params$n[2]))
Type <- factor(Type, c("1", "2"), c("Catalyst 1", "Catalyst 2"))
qqPlot(Yield ~ Type, ylab = "Yield (%)")

# Comparative Box Plots
sp(Yield ~ Type, xlab = "Catalyst type", ylab = "Yield (%)")

# Variance Test
var.test(Cat1, Cat2)
```

```
# --- 5. Two-Sided Test Analysis ---
# ==========================================================================
# CASE: TWO-SIDED TEST
# ==========================================================================
cat("\n== TWO-SIDED TEST ANALYSIS ==\n")
(CritVal_2s <- qt(c(params$alpha/2, 1 - params$alpha/2), df_Case1))
(PVal_2s    <- 2 * (1 - pt(abs(t0), df_Case1)))
(CI_2s      <- -diff(xbar) + CritVal_2s * se_Case1)
t.test(Cat1, Cat2, var.equal = TRUE)
cat(sprintf("Test Statistic (t0): %.3f\nP-Value: %.4f\n", t0, PVal_2s))

cat("Confidence Interval:", "[", CI_2s[1], ",", CI_2s[2], "]\n")

# 5.1 Power at δ = −4, 4:
Pwr.t.test(n_Case1, params$delta, s_Case1, params$alpha)
power.t.test(n = params$n[1], delta = params$delta, sd = s_p,
  sig.level = params$alpha)

# 5.2 Sample Size Requirements (n_1 = n_2 = n)
power.t.test(delta = params$delta[1], sd = s_p, sig.level = params$alpha,
  power = params$pwr.target, type = "two.sample",
  alternative = "two.sided")
power.t.test(delta = params$delta[2], sd = s_p, sig.level = params$alpha,
  power = params$pwr.target)
cat(sprintf("Required n1 = n2 = n for Power ≥ %.2f at δ = %s: 10, 10\n",
      params$pwr.target, paste(params$delta, collapse = ", ")))

power.t.test(n = c(9, 10), delta = params$delta[1], sd = s_p,
  sig.level = params$alph)
power.t.test(n = c(9, 10), delta = params$delta[2], sd = s_p,
  sig.level = params$alph)
```

```
# 5.3 Visualizations
# a. Hypothesis Testing (Current n_1 = n_2 = n = 8)
# Showing the Effect of δ (≡ Δ − Δ₀) on Power
NTplot(t.test(Cat1, Cat2, var.equal = TRUE), mean1 = Delta[1], zaxis = T,
  power = T)
NTplot(t.test(Cat1, Cat2, var.equal = TRUE), mean1 = Delta[2], zaxis = T,
  power = T)

# b. Confidence Interval (Current n_1 = n_2 = n = 8)
NTplot(t.test(Cat1, Cat2, var.equal = TRUE), type = "confidence",
  zaxis = T)

# c. Comparison of Sample Sizes (Targeting Required n)
# Plotting n − 1 vs. required n_1 = n_2 = n = 10 at δ = −4, 4:
power.t.test(delta = params$delta[1], sd = s_p, sig.level = params$alph,
             power = params$pwr.target)
NTplot(power.t.test(n = 9, delta = params$delta[1], sd = s_p,
  sig.level = params$alph), zaxis = T, power = T)
NTplot(power.t.test(n = 10, delta = params$delta[1], sd = s_p,
  sig.level = params$alph), zaxis = T, power = T)

power.t.test(delta = params$delta[2], sd = s_p, sig.level = params$alph,
             power = params$pwr.target)
NTplot(power.t.test(n = 9, delta = params$delta[2], sd = s_p,
  sig.level = params$alph), zaxis = T, power = T)
NTplot(power.t.test(n = 10, delta = params$delta[2], sd = s_p,
  sig.level = params$alph), zaxis = T, power = T)
```

```r
# --- 6. One-Sided (Greater) Test Analysis ---
# ==========================================================================
# CASE: ONE-SIDED (GREATER) TEST
# ==========================================================================
cat("\n=== ONE-SIDED (GREATER) TEST ANALYSIS ===\n")
(CritVal_gr <- qt(c(params$alpha, 1), df_Case1))
(PVal_gr    <- 1 - pt(t0, df_Case1))
(CI_gr      <- -diff(xbar) + CritVal_gr * se_Case1)
t.test(Cat1, Cat2, alternative = "greater", var.equal = TRUE)
cat(sprintf("Test Statistic (t0): %.3f\nP-Value: %.4f\n", t0, PVal_gr))

cat("Confidence Interval:", "[", CI_gr[1], ", ∞)\n")

# 6.1 Power at δ = 4:
Pwr.t.test(n_Case1, params$delta[2], s_Case1, params$alpha, "greater")
power.t.test(n = params$n[1], delta = params$delta[2], sd = s_p,
  sig.level = params$alpha, alternative = "one.sided")

# 6.2 Sample Size Requirements (n_1 = n_2 = n)
power.t.test(delta = params$delta[2], sd = s_p, sig.level = params$alpha,
  power = params$pwr.target, alternative = "one.sided")
cat(sprintf("Required n1 = n2 = n for Power ≥ %.2f at δ = %s: 8\n",
      params$pwr.target, params$delta[2]))

power.t.test(n = c(7, 8), delta = params$delta[2], sd = s_p,
  sig.level = params$alpha, alternative = "one.sided"
)

# 6.3 Visualizations
# a. Hypothesis Testing (Current n_1 = n_2 = n = 8)
# Showing the Effect of δ (≡ Δ − Δ₀) on Power
NTplot(t.test(Cat1, Cat2, alternative = "greater", var.equal = TRUE),
  mean1 = Delta[2], zaxis = T, power = T)

# b. Confidence Interval (Current n_1 = n_2 = n = 8)
NTplot(t.test(Cat1, Cat2, alternative = "greater", var.equal = TRUE),
  type = "confidence", zaxis = T)
```

```
# c. Comparison of Sample Sizes (Targeting Required n)
# Plotting n − 1 vs. required n_1 = n_2 = n = 10 at δ = 4:
power.t.test(delta = params$delta[2], sd = s_p, sig.level = params$alph,
             power = params$pwr.target, alternative = "one.sided")
NTplot(power.t.test(n = 7, delta = params$delta[2], sd = s_p,
  sig.level = params$alpha, alternative = "one.sided"), zaxis = T,
  power = T)
NTplot(power.t.test(n = 8, delta = params$delta[2], sd = s_p,
  sig.level = params$alpha, alternative = "one.sided"), zaxis = T,
  power = T)

# --- 7. One-Sided (Less) Test Analysis ---
# ==========================================================================
# CASE: ONE-SIDED (LESS) TEST
# ==========================================================================
cat("\n=== ONE-SIDED (LESS) TEST ANALYSIS ===\n")
(CritVal_le <- qt(c(0, 1 - params$alpha), df_Case1))
(PVal_le    <- pt(t0, df_Case1))
(CI_le      <- -diff(xbar) + CritVal_le * se_Case1)
t.test(Cat1, Cat2, alternative = "less", var.equal = TRUE)
cat(sprintf("Test Statistic (t0): %.3f\nP-Value: %.4f\n", t0, PVal_le))

cat("Confidence Interval:", "(−∞,", CI_le[2], "]\n")

# 7.1 Power at δ = −4:
Pwr.t.test(n_Case1, params$delta[1], s_Case1, params$alpha, "less")
power.t.test(n = params$n[1], delta = abs(params$delta[1]), sd = s_p,
  sig.level = params$alpha, alternative = "one.sided")

# 7.2 Sample Size Requirements (n_1 = n_2 = n)
power.t.test(delta = abs(params$delta[1]), sd = s_p,
  sig.level = params$alpha, power = params$pwr.target,
  alternative = "one.sided")
cat(sprintf("Required n1 = n2 = n for Power ≥ %.2f at δ = %s: 8\n",
      params$pwr.target, params$delta[1]))
```

```
power.t.test(n = c(7, 8), delta = abs(params$delta[1]), sd = s_p,
  sig.level = params$alph, alternative = "one.sided")

# 7.3 Visualizations
# a. Hypothesis Testing (Current n_1 = n_2 = n = 8)
# Showing the Effect of δ (≡ Δ − Δ₀) on Power
NTplot(t.test(Cat1, Cat2, alternative = "less", var.equal = TRUE),
  mean1 = Delta[1], zaxis = T, power = T)

# b. Confidence Interval (Current n_1 = n_2 = n = 8)
NTplot(t.test(Cat1, Cat2, alternative = "less", var.equal = TRUE),
  type = "confidence", zaxis = T)

# c. Comparison of Sample Sizes (Targeting Required n)
# Plotting n − 1 vs. required n_1 = n_2 = n = 10 at δ = −4, 4:
power.t.test(delta = abs(params$delta[1]), sd = s_p,
  sig.level = params$alph, power = params$pwr.target,
  alternative = "one.sided")
NTplot(power.t.test(n = 7, delta = abs(params$delta[1]), sd = s_p,
  sig.level = params$alph, alternative = "one.sided"), zaxis = T,
  power = T)
NTplot(power.t.test(n = 8, delta = abs(params$delta[1]), sd = s_p,
  sig.level = params$alpha, alternative = "one.sided"), zaxis = T,
  power = T)

# Case 1: σ_1² ≠ σ_2²

# EXAMPLE 5-5 Arsenic in Drinking Water

# Figure 5-3 Normal probability of the arsenic concentration data from
# Example 5-5.

# --- 3*. Data & Parameters ---
(PHX     <- c( 3,  7, 25, 10, 15,  6, 12, 25, 15,  7))
(RuralAZ <- c(48, 44, 40, 38, 33, 21, 20, 12,  1, 18))
(params <- list(Delta0 = 0, alpha = 0.05,
            n = c(length(PHX), length(RuralAZ))))
```

```
# --- 4*. Basic Stats, Normality Test, Box Plots & Variance Test ---
summary(PHX)
summary(RuralAZ)
(xbar <- c(mean(PHX), mean(RuralAZ)))
(s <- c(sd(PHX), sd(RuralAZ)))
(df <- params$n - 1)
(df_Case2 <- sum(s^2/params$n)^2 / sum((s^2/params$n)^2/(params$n - 1)))
(se_Case2 <- sqrt(sum(s^2 / params$n)))
(tstar0 <- (-diff(xbar) - params$Delta0) / se_Case2)

# Normality Test
shapiro.test(PHX)
shapiro.test(RuralAZ)
Conc <- c(PHX, RuralAZ)
Type <- c(rep(1, params$n[1]), rep(2, params$n[2]))
Type <- factor(Type, c("1", "2"), c("PHX", "RuralAZ"))
qqPlot(Conc ~ Type, ylab = "Concentration (ppb)")

# Comparative Box Plots
sp(Conc ~ Type, xlab = "Community type", ylab = "Concentration (ppb)")

# Variance Test
var.test(PHX, RuralAZ)

# --- 5*. Two-Sided Test Analysis ---
# ==========================================================================
# CASE: TWO-SIDED TEST
# ==========================================================================
cat("\n=== TWO-SIDED TEST ANALYSIS ===\n")
(CritVal_2s <- qt(c(params$alpha/2, 1 - params$alpha/2), df_Case2))
(PVal_2s    <- 2 * (1 - pt(abs(tstar0), df_Case2)))
(CI_2s      <- -diff(xbar) + CritVal_2s * se_Case2)
t.test(PHX, RuralAZ)
cat(sprintf("Test Statistic (t*0): %.3f\nP-Value: %.4f\n", tstar0,
PVal_2s))

cat("Confidence Interval:", "[", CI_2s[1], ",", CI_2s[2], "]\n")
```

```
# 5*.1 Visualizations
# a. Hypothesis Testing
NTplot(t.test(PHX, RuralAZ), zaxis = T)

# b. Confidence Interval
NTplot(t.test(PHX, RuralAZ), type = "confidence", zaxis = T)

# --- 6*. One-Sided (Greater) Test Analysis ---
# ========================================================================
# CASE: ONE-SIDED (GREATER) TEST
# ========================================================================
cat("\n=== ONE-SIDED (GREATER) TEST ANALYSIS ===\n")
(CritVal_gr <- qt(c(params$alpha, 1), df_Case2))
(PVal_gr    <- 1 - pt(tstar0, df_Case2))
(CI_gr      <- -diff(xbar) + CritVal_gr * se_Case2)
t.test(PHX, RuralAZ, alternative = "greater")
cat(sprintf("Test Statistic (t*0): %.3f\nP-Value: %.4f\n", tstar0,
PVal_gr))

cat("Confidence Interval:", "[", CI_gr[1], ", ∞)\n")

# 6*.1 Visualizations
# a. Hypothesis Testing
NTplot(t.test(PHX, RuralAZ, alternative = "greater"), zaxis = T)

# b. Confidence Interval
NTplot(t.test(PHX, RuralAZ, alternative = "greater"), type = "confidence",
  zaxis = T)
```

```
# --- 7*. One-Sided (Less) Test Analysis ---
# ==========================================================================
# CASE: ONE-SIDED (LESS) TEST
# ==========================================================================
cat("\n=== ONE-SIDED (LESS) TEST ANALYSIS ===\n")
(CritVal_le <- qt(c(0, 1 - params$alpha), df_Case2))
(PVal_le    <- pt(tstar0, df_Case2))
(CI_le      <- -diff(xbar) + CritVal_le * se_Case2)
t.test(PHX, RuralAZ, alternative = "less")
cat(sprintf("Test Statistic (t*0): %.3f\nP-Value: %.4f\n", tstar0,
PVal_le))

cat("Confidence Interval:", "(−∞,", CI_le[2], "]\n")

# 7*.1 Visualizations
# a. Hypothesis Testing
NTplot(t.test(PHX, RuralAZ, alternative = "less"), zaxis = T)

# b. Confidence Interval
NTplot(t.test(PHX, RuralAZ, alternative = "less"), type = "confidence",
  zaxis = T)

# § 5-3.2 Type II Error and Choice of Sample Size

# Case 1: σ_1² = σ_2² = σ²

# EXAMPLE 5-6 Chemical Process Yield

# Refer to EXAMPLE 5-4.

# § 5-3.3 Confidence Interval on the Difference in Means

# Case 1: σ_1² = σ_2² = σ²

# EXAMPLE 5-7 Calcium in Doped Cement

# Refer to EXAMPLE 5-4.
```

```
# --- 3**. Parameters ---
(params <- list(Delta0 = 0, alpha = 0.05, n = c(10, 15)))

# --- 4**. Basic Stats & Data Simulation ---
(xbar <- c(90.0, 87.0))
(s <- c(5.0, 4.0))
(df <- params$n - 1)
(df_Case1 <- sum(df))
(s_p <- sqrt(sum(df * s^2) / df_Case1))
(se_Case1 <- s_p * sqrt(sum(1 / params$n)))
(t0 <- (-diff(xbar) - params$Delta0) / se_Case1)

set.seed(123)
Std_raw <- rnorm(params$n[1])

# Rescale to have sample mean xbar[1] and sample standard deviation s[1]
Std <- xbar[1] + s[1] * (Std_raw - mean(Std_raw)) / sd(Std_raw)

Lead_raw <- rnorm(params$n[2])

# Rescale to have sample mean xbar[2] and sample standard deviation s[2]
Lead <- xbar[2] + s[2] * (Lead_raw - mean(Lead_raw)) / sd(Lead_raw)

# --- 5**. Two-Sided Test Analysis ---
# ==========================================================================
# CASE: TWO-SIDED TEST
# ==========================================================================
cat("\n=== TWO-SIDED TEST ANALYSIS ===\n")
(CritVal_2s <- qt(c(params$alpha/2, 1 - params$alpha/2), df_Case1))
(PVal_2s    <- 2 * (1 - pt(abs(t0), df_Case1)))
(CI_2s      <- -diff(xbar) + CritVal_2s * se_Case1)
t.test(Std, Lead, var.equal = TRUE)
cat(sprintf("Test Statistic (t0): %.3f\nP-Value: %.4f\n", t0, PVal_2s))

cat("Confidence Interval:", "[", CI_2s[1], ",", CI_2s[2], "]\n")
```

```
# 5**.1 Visualizations
# a. Hypothesis Testing
NTplot(t.test(Std, Lead, var.equal = TRUE), zaxis = T)

# b. Confidence Interval (Current n_1 = n_2 = n = 8)
NTplot(t.test(Std, Lead, var.equal = TRUE), type = "confidence", zaxis = T)

# --- 6**. One-Sided (Greater) Test Analysis ---
# ========================================================================
# CASE: ONE-SIDED (GREATER) TEST
# ========================================================================
cat("\n=== ONE-SIDED (GREATER) TEST ANALYSIS ===\n")
(CritVal_gr <- qt(c(params$alpha, 1), df_Case1))
(PVal_gr    <- 1 - pt(t0, df_Case1))
(CI_gr      <- -diff(xbar) + CritVal_gr * se_Case1)
t.test(Std, Lead, alternative = "greater", var.equal = TRUE)
cat(sprintf("Test Statistic (t0): %.3f\nP-Value: %.4f\n", t0, PVal_gr))

cat("Confidence Interval:", "[", CI_gr[1], ", ∞)\n")

# 6**.1 Visualizations
# a. Hypothesis Testing
NTplot(t.test(Std, Lead, alternative = "greater", var.equal = TRUE),
  zaxis = T)

# b. Confidence Interval
NTplot(t.test(Std, Lead, alternative = "greater", var.equal = TRUE),
  type = "confidence", zaxis = T)
```

```
# --- 7**. One-Sided (Less) Test Analysis ---
# ==========================================================================
# CASE: ONE-SIDED (LESS) TEST
# ==========================================================================
cat("\n=== ONE-SIDED (LESS) TEST ANALYSIS ===\n")
(CritVal_le <- qt(c(0, 1 - params$alpha), df_Case1))
(PVal_le    <- pt(t0, df_Case1))
(CI_le      <- -diff(xbar) + CritVal_le * se_Case1)
t.test(Std, Lead, alternative = "less", var.equal = TRUE)
cat(sprintf("Test Statistic (t0): %.3f\nP-Value: %.4f\n", t0, PVal_le))

cat("Confidence Interval:", "(−∞,", CI_le[2], "]\n")

# 7**.1. Visualizations
# a. Hypothesis Testing
NTplot(t.test(Std, Lead, alternative = "less", var.equal = TRUE),
  zaxis = T)

# b. Confidence Interval
NTplot(t.test(Std, Lead, alternative = "less", var.equal = TRUE),
  type = "confidence", zaxis = T)

# EXERCISES FOR § 5-3
```

```
# § 5-4 The Paired t-Test

# Refer to § 4-5.

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(car, HH)

# EXAMPLE 5-8 Shear Strengh of Steel Girders

# Table 5-3 Strength Predictions for Nine Steel Plate Girders (Predicted
# Load/Observed Load)

Karlsruhe <- c(1.186, 1.151, 1.322, 1.339, 1.200, 1.402, 1.365, 1.537,
1.559)

Lehigh <- c(1.061, 0.992, 1.063, 1.062, 1.065, 1.178, 1.037, 1.086, 1.052)

(KL.diff <- Karlsruhe - Lehigh)

ObsNo. <- 1:length(KL.diff); ShearStr <- data.frame(KL.diff, ObsNo.)

summary(ShearStr); spm(ShearStr)

shapiro.test(KL.diff); qqp(KL.diff)

(dbar <- mean(KL.diff)); (s <- sd(KL.diff)); (Delta0 <- 0)

(sig.lv <- .05); (n <- length(KL.diff)); (df <- n - 1); (se <- s/sqrt(n))

(t0 <- (dbar - Delta0)/se)

# alt = "two.sided":

t.test(Karlsruhe, Lehigh, paired = TRUE)

(CritVal <- qt(c(sig.lv/2, 1 - sig.lv/2), df))
```

```
(PVal <- 2*(1 - pt(abs(t0), df)))

(CI <- dbar + CritVal*se)

NTplot(t.test(Karlsruhe, Lehigh, paired = TRUE), zaxis = T, power = T)

NTplot(t.test(Karlsruhe, Lehigh, paired = TRUE), "c", zaxis = T)

# alt = "gr":

t.test(Karlsruhe, Lehigh, alternative = "greater", paired = TRUE)

(CritVal <- qt(1 - sig.lv, df))

(PVal <- 1 - pt(t0, df))

(CI <- c(dbar - CritVal*se, Inf))

NTplot(t.test(Karlsruhe, Lehigh, alternative = "greater", paired = TRUE),
  zaxis = T, power = T)

NTplot(t.test(Karlsruhe, Lehigh, alternative = "greater", paired = TRUE),
  "c", zaxis = T)

# alt = "less":

t.test(Karlsruhe, Lehigh, alternative = "less", paired = TRUE)

(CritVal <- qt(sig.lv, df))

(PVal <- pt(t0, df))

(CI <- c(-Inf, dbar - CritVal*se))

NTplot(t.test(Karlsruhe, Lehigh, alternative = "less", paired = TRUE),
  zaxis = T, power = T)
```

```
NTplot(t.test(Karlsruhe, Lehigh, alternative = "less", paired = TRUE), "c",
  zaxis = T)

# EXAMPLE 5-9 Parallel Parking

# Refer to EXAMPLE 5-8.

# Table 5-4 Time in Seconds to Parallel Park Two Automobiles

Auto1 <- c(37.0, 25.8, 16.2, 24.2, 22.0, 33.4, 23.8, 58.2, 33.6, 24.4,
23.4, 21.2, 36.2, 29.8)

Auto2 <- c(17.8, 20.2, 16.8, 41.4, 21.4, 38.4, 16.8, 32.2, 27.8, 23.2,
29.6, 20.6, 32.2, 53.8)

(Auto.diff <- Auto1 - Auto2)

ObsNo. <- 1:length(Auto.diff); ParP <- data.frame(Auto.diff, ObsNo.)

summary(ParP); spm(ParP)

shapiro.test(Auto.diff); qqp(Auto.diff)

(dbar <- mean(Auto.diff)); (s <- sd(Auto.diff)); (Delta0 <- 0)

(sig.lv <- .1); (n <- length(Auto.diff)); (df <- n - 1); (se <- s/sqrt(n))

# alt = "two.sided":

t.test(Auto1, Auto2, paired = TRUE, conf.level = 0.9)

(CritVal <- qt(c(sig.lv/2, 1 - sig.lv/2), df))

(CI <- dbar + CritVal*se)

NTplot(t.test(Auto1, Auto2, paired = TRUE, conf.level = 0.9),zaxis = T,
  power = T)
```

```
NTplot(t.test(Auto1, Auto2, paired = TRUE, conf.level = 0.9), "c",
  zaxis = T)

# alt = "gr":

NTplot(t.test(Auto1, Auto2, alternative = "greater", paired = TRUE,
  conf.level = 0.9),zaxis = T, power = T)

NTplot(t.test(Auto1, Auto2, alternative = "greater", paired = TRUE,
  conf.level = 0.9), "c", zaxis = T)

# alt = "less":

NTplot(t.test(Auto1, Auto2, alternative = "less", paired = TRUE,
  conf.level = 0.9),zaxis = T, power = T)

NTplot(t.test(Auto1, Auto2, alternative = "less", paired = TRUE,
  conf.level = 0.9), "c", zaxis = T)

# EXERCISES FOR § 5-4
```

```
# § 5-5 Inference on the Ratio of Variances of Two Normal Populations

# Refer to § 4-6.

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(car, HH)

# § 5-5.1 Hypothesis Testing on the Ratio of Two Variances

# Figure 5-4 Probability density functions of two F distributions.

old.omd <- par(omd = c(.07, .88, 0, 1))

F.setup(5, 15, xlim.in = c(0, qf(.95, 5, 5)),
main.in = expression("F density:" ~~ nu[1]=="5" ~~ nu[2]=="5, 15"))

F.curve(5, 5); F.curve(5, 15, col = 2)

F.setup(xlim.in = c(0, qf(.95, 2, 2)), ylim.in = c(0, 1.5), main.in =
expression("F density:" ~~ nu[1]=="1, 2, …, 10" ~~ nu[2]==nu[1]))

for (i in 1:10) F.curve(i, i, f = seq(0, par()$usr[2], length = 1001),
col = i)

F.setup(20, 20, xlim.in = c(0, qf(.95, 11, 11)), main.in =
expression("F density:" ~~ nu[1]=="11, 12, …, 20" ~~ nu[2]==nu[1]))

for (i in 11:20) F.curve(i, i, col = i - 10)

F.setup(30, 30, xlim.in = c(0, qf(.95, 21, 21)), main.in =
expression("F density:" ~~ nu[1]=="21, 22, …, 30" ~~ nu[2]==nu[1]))

for (i in 21:30) F.curve(i, i, col = i - 20)

F.setup(120, 120, xlim.in = c(0, qf(.95, 10, 10)), main.in =
expression("F density:" ~~ nu[1]=="10, 20, …, 120" ~~ nu[2]==nu[1]))
```

```
for (i in 10*(1:12)) F.curve(i, i, col = i/10)

par(old.omd)

# Figure 5-5 Upper and lower percentage points of the F distribution.

# Figure 5-6 The F distribution for the test of H0: sigma1^2 = sigma2^2
# with critical region values for (a) H1:sigma1^2 not = sigma2^2, (b) H1:
# sigma1^2 > sigma2^2, and (c) H1: sigma1^2 < sigma2^2.

Pwr.var.test.t <- function (n, rto0, rto1, sig.lv) {
df <- n - 1
1 - pf(qf(1 - sig.lv/2, df[1], df[2])*rto0/rto1, df[1], df[2]) +
pf(qf(sig.lv/2, df[1], df[2])*rto0/rto1, df[1], df[2])}

Pwr.var.test.g <- function (n, rto0, rto1, sig.lv) {
df <- n - 1; 1 - pf(qf(1 - sig.lv, df[1], df[2])*rto0/rto1, df[1], df[2])}

Pwr.var.test.l <- function (n, rto0, rto1, sig.lv) {
df <- n - 1; pf(qf(sig.lv, df[1], df[2])*rto0/rto1, df[1], df[2])}

# EXAMPLE 5-10 Etching Wafers

(n <- c(16, 16)); (s <- c(1.96, 2.13)); (rto0 <- 1); (sig.lv <- .05)

(f0 <- (s[1]^2/s[2]^2)/rto0); (df <- n - 1); (rto1 <- c(2^2, .5^2))

x1.0 <- rnorm(n[1]); x1 <- s[1]*(x1.0 - mean(x1.0))/sd(x1.0)

# x1 is a simulated normal data set with sample size n[1] and sample
# standard deviation s[1].

length(x1); sd(x1)

x2.0 <- rnorm(n[2]); x2 <- s[2]*(x2.0 - mean(x2.0))/sd(x2.0)
```

```
# x2 is a simulated normal data set with sample size n[2] and sample
# standard deviation s[2].

length(x2); sd(x2)

# alt = "two.sided":

var.test(x1, x2)

(CritVal <- qf(c(sig.lv/2, 1 - sig.lv/2), df[1], df[2]))

(PVal <- 2*min(pf(f0, df[1], df[2]), 1 - pf(f0, df[1], df[2])))

Pwr.var.test.t(n, rto0, rto1, sig.lv)

(CI <- f0/qf(c(1 - sig.lv/2, sig.lv/2), df[1], df[2]))

(qf <- range(c(f0, qf(1 - pf(f0, df[1], df[2]), df[1], df[2]))))

old.omd <- par(omd = c(.07, .88, .05, 1))

F.setup(df[1], df[2])

F.curve(df[1], df[2], 0, F, c(sig.lv/2, sig.lv/2), col = 'blue')

F.observed(qf[1], "green", df[1], df[2], 0, F, "f", "left")

F.observed(qf[2], "green", df[1], df[2])

axis(3, 2.5, expression(n[1]==nu[1] + 1 ~~ n[2]==nu[2] + 1 ~~ s[1]==1.96 ~~
s[2]==2.13 ~~ f[0]==s[1]^2/s[2]^2), T, 1.5, lwd = 0)

axis(3, 2.5, expression(LCL ~~ UCL ==f[0]/f[nu[1]~nu[2]]), T, -2.5,
lwd = 0)

axis(1, 5.2, round(PVal, 4), T, 3.5, lwd = 0)
```

```
axis(1, -.2, expression(f[0]/f[nu[1]~nu[2]]), T, 5, lwd = 0)

axis(1, qf(c(sig.lv/2, 1 - sig.lv/2), df[1], df[2]),
round(f0/qf(c(sig.lv/2, 1 - sig.lv/2), df[1], df[2]), 4), T, 5, lwd = 0)

par(old.omd)

# alt = "gr":

var.test(x1, x2, rto0, "g")

(CritVal <- qf(1 - sig.lv, df[1], df[2]))

(PVal <- 1 - pf(f0, df[1], df[2]))

Pwr.var.test.g(n, rto0, rto1[1], sig.lv)

(CI <- c(f0/qf(1 - sig.lv, df[1], df[2]), Inf))

old.omd <- par(omd = c(.07, .88, .05, 1))

F.setup(df[1], df[2])

F.curve(df[1], df[2], 0, F, sig.lv, col = 'blue')

F.observed(f0, "green", df[1], df[2])

axis(3, 2.5, expression(n[1]==nu[1] + 1 ~~ n[2]==nu[2] + 1 ~~ s[1]==1.96 ~~
s[2]==2.13 ~~ f[0]==s[1]^2/s[2]^2), T, 1.5, lwd = 0)

axis(3, 2.5, expression(LCL ==f[0]/f[nu[1]~nu[2]]), T, -2.5, lwd = 0)

axis(1, 5.2, round(PVal, 4), T, 3.5, lwd = 0)

axis(1, -.2, expression(f[0]/f[nu[1]~nu[2]]), T, 5, lwd = 0)
```

```
axis(1, qf(1 - sig.lv, df[1], df[2]),
round(f0/qf(1 - sig.lv, df[1], df[2]), 4), T, 5, lwd = 0)

par(old.omd)

# alt = "less":

var.test(x1, x2, rto0, "l")

(CritVal <- qf(sig.lv, df[1], df[2]))

(PVal <- pf(f0, df[1], df[2]))

Pwr.var.test.l(n, rto0, rto1[2], sig.lv)

(CI <- c(0, f0/qf(sig.lv, df[1], df[2])))

old.omd <- par(omd = c(.07, .88, .05, 1))

F.setup(df[1], df[2])

F.curve(df[1], df[2], 0, F, 1 - sig.lv, shade = "l", col = 'blue')

F.observed(f0, "green", df[1], df[2], 0, F, "f", "left")

axis(3, 2.5, expression(n[1]==nu[1] + 1 ~~ n[2]==nu[2] + 1 ~~ s[1]==1.96 ~~
s[2]==2.13 ~~ f[0]==s[1]^2/s[2]^2), T, 1.5, lwd = 0)

axis(3, 2.5, expression(UCL ==f[0]/f[nu[1]~nu[2]]), T, -2.5, lwd = 0)

axis(1, 5.2, round(PVal, 4), T, 3.5, lwd = 0)

axis(1, -.2, expression(f[0]/f[nu[1]~nu[2]]), T, 5, lwd = 0)

axis(1, qf(sig.lv, df[1], df[2]), round(f0/qf(sig.lv, df[1], df[2]), 4), T,
5, lwd = 0)
```

```
par(old.omd)

# § 5-5.2 Confidence Interval on the Ratio of Two Variances

# EXAMPLE 5-11 Surface Finish

# Refer to EXAMPLE 5-10.

(n <- c(11, 16)); (s <- c(5.1, 4.7)); (rto0 <- 1); (sig.lv <- .1)

(f0 <- (s[1]^2/s[2]^2)/rto0); (df <- n - 1)

x1.0 <- rnorm(n[1]); x1 <- s[1]*(x1.0 - mean(x1.0))/sd(x1.0)

# x1 is a simulated normal data set with sample size n[1] and sample
# standard deviation s[1].

length(x1); sd(x1)

x2.0 <- rnorm(n[2]); x2 <- s[2]*(x2.0 - mean(x2.0))/sd(x2.0)

# x2 is a simulated normal data set with sample size n[2] and sample
# standard deviation s[2].

length(x2); sd(x2)

# alt = "two.sided":

var.test(x1, x2, rto0, "t", 1 - sig.lv)

(CritVal <- qf(c(sig.lv/2, 1 - sig.lv/2), df[1], df[2]))

(PVal <- 2*min(pf(f0, df[1], df[2]), 1 - pf(f0, df[1], df[2])))

(CI <- f0/qf(c(1 - sig.lv/2, sig.lv/2), df[1], df[2]))

(qf <- range(c(f0, qf(1 - pf(f0, df[1], df[2]), df[1], df[2]))))
```

```
old.omd <- par(omd = c(.07, .88, .05, 1))

F.setup(df[1], df[2])

F.curve(df[1], df[2], 0, F, c(sig.lv/2, sig.lv/2), col = 'blue')

F.observed(qf[1], "green", df[1], df[2], 0, F, "f", "left")

F.observed(qf[2], "green", df[1], df[2])

axis(3, 2.5, expression(n[1]==nu[1] + 1 ~~ n[2]==nu[2] + 1 ~~ s[1]==1.96 ~~
s[2]==2.13 ~~ f[0]==s[1]^2/s[2]^2), T, 1.5, lwd = 0)

axis(3, 2.5, expression(LCL ~~ UCL ==f[0]/f[nu[1]~nu[2]]), T, -2.5,
lwd = 0)

axis(1, 5.2, round(PVal, 4), T, 3.5, lwd = 0)

axis(1, -.2, expression(f[0]/f[nu[1]~nu[2]]), T, 5, lwd = 0)

axis(1, qf(c(sig.lv/2, 1 - sig.lv/2), df[1], df[2]),
round(f0/qf(c(sig.lv/2, 1 - sig.lv/2), df[1], df[2]), 4), T, 5, lwd = 0)

par(old.omd)

# EXERCISES FOR § 5-5
```

```
# § 5-6 Inference on Two Population Proportions

# Refer to § 4-7.

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(car, HH)

# § 5-6.1 Hypothesis Testing on the Equality of Two Binomial Proportions

# EXAMPLE 5-12 Interocular Lenses

(n <- c(300, 300)); (x <- c(253, 196)); (Delta0 <- 0); (aprx.lv <- .05)

(phat <- x/n); (qhat <- 1 - phat); (phat.diff <- -diff(phat))

(phat.p <- sum(x)/sum(n)); (qhat.p <- 1 - phat.p)

(se.p <- sqrt(phat.p*qhat.p*sum(1/n)))

(z0 <- phat.diff/se.p)

(n0 <- mean(n)); (sd.p <- sqrt(n0)*se.p)

(sehat <- sqrt(sum(phat*qhat/n))); (sdhat <- sqrt(n0)*sehat)

(ntilde <- n + 2); (xtilde <- x + 1)

(ptilde <- xtilde/ntilde); (qtilde <- 1 - ptilde)

(ptilde.diff <- -diff(ptilde))

(setilde <- sqrt(sum(ptilde*qtilde/ntilde)))

(n0tilde <- mean(ntilde)); (sdtilde <- sqrt(n0tilde)*setilde)

# alt = "two.sided":
```

```
prop.test(x, n, NULL, "t", 1 - aprx.lv, F) # Aprx

(CritVal <- qnorm(c(aprx.lv/2, 1 - aprx.lv/2))) # Aprx

(PVal <- 2*(1 - pnorm(abs(z0)))) # Aprx

(AsymptoticCI <- phat.diff + CritVal*sehat) # Aprx

prop.test(xtilde, ntilde, NULL, "t", 1 - aprx.lv, F) # Aprx

(AgrestiCaffoCI <- ptilde.diff + CritVal*setilde) # Aprx

normal.and.t.dist(Delta0, NA, phat.diff, sd.p, n0, NA, aprx.lv/2,
aprx.lv/2, F, T, T) # Aprx

normal.and.t.dist(NA, NA, phat.diff, sdhat, n0, NA, aprx.lv/2, aprx.lv/2,
F, F, T, T, "c") # Aprx

normal.and.t.dist(NA, NA, ptilde.diff, sdtilde, n0tilde, NA, aprx.lv/2,
aprx.lv/2, F, F, T, T, "c") # Aprx

# alt = "gr":

prop.test(x, n, NULL, "g", 1 - aprx.lv, F) # Aprx

(CritVal <- qnorm(1 - aprx.lv)) # Aprx

(PVal <- 1 - pnorm(z0)) # Aprx

(AsymptoticCI <- c(phat.diff - CritVal*sehat, 1)) # Aprx

prop.test(xtilde, ntilde, NULL, "g", 1 - aprx.lv, F) # Aprx

(AgrestiCaffoCI <- c(ptilde.diff - CritVal*setilde, 1)) # Aprx

normal.and.t.dist(Delta0, NA, phat.diff, sd.p, n0, NA, aprx.lv, aprx.lv, F,
T) # Aprx
```

```
normal.and.t.dist(NA, NA, phat.diff, sdhat, n0, NA, aprx.lv, aprx.lv, F, F,
T, F, "c") # Aprx

normal.and.t.dist(NA, NA, ptilde.diff, sdtilde, n0tilde, NA, aprx.lv,
aprx.lv, F, F, T, F, "c") # Aprx

# alt = "less":

prop.test(x, n, NULL, "l", 1 - aprx.lv, F) # Aprx

(CritVal <- qnorm(aprx.lv)) # Aprx

(PVal <- pnorm(z0)) # Aprx

(AsymptoticCI <- c(-1, phat.diff - CritVal*sehat)) # Aprx

prop.test(xtilde, ntilde, NULL, "l", 1 - aprx.lv, F) # Aprx

(AgrestiCaffoCI <- c(-1, ptilde.diff - CritVal*setilde)) # Aprx

normal.and.t.dist(Delta0, NA, phat.diff, sd.p, n0, NA, aprx.lv, aprx.lv, F,
T, T, F) # Aprx

normal.and.t.dist(NA, NA, phat.diff, sdhat, n0, NA, aprx.lv, aprx.lv, F, F,
F, T, "c") # Aprx

normal.and.t.dist(NA, NA, ptilde.diff, sdtilde, n0tilde, NA, aprx.lv,
aprx.lv, F, F, F, T, "c") # Aprx

# § 5-6.2 Type II Error and Choice of Sample Size
```

```
AprxPwr.prop.test <- function (n, p, aprx.lv, alt) {
Delta <- -diff(p); pbar <- sum(n*p)/sum(n)
se0 <- sqrt(pbar*(1 - pbar)*sum(1/n)); se1 <- sqrt(sum(p*(1 - p)/n))
m <- Delta/se0; sd <- se1/se0
if (alt == 2) 1 - pnorm(qnorm(1 - aprx.lv/2), m, sd) +
pnorm(qnorm(aprx.lv/2), m, sd)
else 1 - pnorm(qnorm(1 - aprx.lv), abs(m), sd)}

AprxSplSz.prop.test <- function (p, aprx.lv, pwr.at.least, alt)
ceiling((qnorm(1 - aprx.lv/alt)*sqrt(sum(p)*sum(1 - p)/2) +
qnorm(pwr.at.least)*sqrt(sum(p*(1 - p))))^2/diff(p)^2)

# Refer to EXAMPLE 5-12.

(n <- c(300, 300)); (Delta0 <- 0); (aprx.lv <- .05); (n0 <- mean(n))

(pwr.at.least <- .9)

# alt = "two.sided":

(p <- c(.8, .7)); (q <- 1 - p); (Delta <- -diff(p))

(pbar <- sum(n*p)/sum(n)); (qbar <- 1 - pbar)

(se0 <- sqrt(pbar*qbar*sum(1/n))); (se <- sqrt(sum(p*q/n)))

AprxPwr.prop.test(n, p, aprx.lv, 2)

power.prop.test(n0, p[1], p[2], aprx.lv, NULL, "t", T) # Aprx

(n.2 <- AprxSplSz.prop.test(p, aprx.lv, pwr.at.least, 2))

power.prop.test(NULL, p[1], p[2], aprx.lv, pwr.at.least, "t", T) # Aprx

AprxPwr.prop.test(c(n.2 - 1, n.2 - 1), p, aprx.lv, 2) # Aprx

power.prop.test(n.2 - 1, p[1], p[2], aprx.lv, NULL, "t", T) # Aprx
```

```
AprxPwr.prop.test(c(n.2, n.2), p, aprx.lv, 2)

power.prop.test(n.2, p[1], p[2], aprx.lv, NULL, "t", T) # Aprx

(p <- c(.7, .8)); (q <- 1 - p); (Delta <- -diff(p))

(pbar <- sum(n*p)/sum(n)); (qbar <- 1 - pbar)

(se0 <- sqrt(pbar*qbar*sum(1/n))); (se <- sqrt(sum(p*q/n)))

AprxPwr.prop.test(n, p, aprx.lv, 2)

power.prop.test(n0, p[1], p[2], aprx.lv, NULL, "t", T) # Aprx

(n.2 <- AprxSplSz.prop.test(p, aprx.lv, pwr.at.least, 2))

power.prop.test(NULL, p[1], p[2], aprx.lv, pwr.at.least, "t", T) # Aprx

AprxPwr.prop.test(c(n.2 - 1, n.2 - 1), p, aprx.lv, 2) # Aprx

power.prop.test(n.2 - 1, p[1], p[2], aprx.lv, NULL, "t", T) # Aprx

AprxPwr.prop.test(c(n.2, n.2), p, aprx.lv, 2)

power.prop.test(n.2, p[1], p[2], aprx.lv, NULL, "t", T) # Aprx

# alt = "gr":

(p <- c(.8, .7)); (q <- 1 - p); (Delta <- -diff(p))

(pbar <- sum(n*p)/sum(n)); (qbar <- 1 - pbar)

(se0 <- sqrt(pbar*qbar*sum(1/n))); (se <- sqrt(sum(p*q/n)))

AprxPwr.prop.test(n, p, aprx.lv, 1)

power.prop.test(n0, p[1], p[2], aprx.lv, NULL, "o") # Aprx
```

```
(n.1 <- AprxSplSz.prop.test(p, aprx.lv, pwr.at.least, 1))

power.prop.test(NULL, p[1], p[2], aprx.lv, pwr.at.least, "o") # Aprx

AprxPwr.prop.test(c(n.1 - 1, n.1 - 1), p, aprx.lv, 1)

power.prop.test(n.1 - 1, p[1], p[2], aprx.lv, NULL, "o") # Aprx

AprxPwr.prop.test(c(n.1, n.1), p, aprx.lv, 1)

power.prop.test(n.1, p[1], p[2], aprx.lv, NULL, "o") # Aprx

# alt = "less":

(p <- c(.7, .8)); (q <- 1 - p); (Delta <- -diff(p))

(pbar <- sum(n*p)/sum(n)); (qbar <- 1 - pbar)

(se0 <- sqrt(pbar*qbar*sum(1/n))); (se <- sqrt(sum(p*q/n)))

AprxPwr.prop.test(n, p, aprx.lv, 1)

power.prop.test(n0, p[1], p[2], aprx.lv, NULL, "o") # Aprx

(n.1 <- AprxSplSz.prop.test(p, aprx.lv, pwr.at.least, 1))

power.prop.test(NULL, p[1], p[2], aprx.lv, pwr.at.least, "o") # Aprx

AprxPwr.prop.test(c(n.1 - 1, n.1 - 1), p, aprx.lv, 1)

power.prop.test(n.1 - 1, p[1], p[2], aprx.lv, NULL, "o") # Aprx

AprxPwr.prop.test(c(n.1, n.1), p, aprx.lv, 1)

power.prop.test(n.1, p[1], p[2], aprx.lv, NULL, "o") # Aprx

# § 5-6.3 Confidence Interval on the Difference in Binomial Proportions
```

```
# EXAMPLE 5-13 Crankshaft Bearings

# Refer to EXAMPLES 4-14 and 5-12.

(n <- c(85, 85)); (x <- c(10, 8)); (Delta0 <- 0); (aprx.lv <- .05)

(phat <- x/n); (qhat <- 1 - phat); (phat.diff <- -diff(phat))

(sehat <- sqrt(sum(phat*qhat/n)))

(n0 <- mean(n)); (sdhat <- sqrt(n0)*sehat)

(ntilde <- n + 2); (xtilde <- x + 1)

(ptilde <- xtilde/ntilde); (qtilde <- 1 - ptilde)

(ptilde.diff <- -diff(ptilde))

(setilde <- sqrt(sum(ptilde*qtilde/ntilde)))

(n0tilde <- mean(ntilde)); (sdtilde <- sqrt(n0tilde)*setilde)

# alt = "two.sided":

prop.test(x, n, NULL, "t", 1 - aprx.lv, F) # Aprx

(CritVal <- qnorm(c(aprx.lv/2, 1 - aprx.lv/2))) # Aprx

(AsymptoticCI <- phat.diff + CritVal*sehat) # Aprx

prop.test(xtilde, ntilde, NULL, "t", 1 - aprx.lv, F) # Aprx

(AgrestiCaffoCI <- ptilde.diff + CritVal*setilde) # Aprx

normal.and.t.dist(NA, NA, phat.diff, sdhat, n0, NA, aprx.lv/2, aprx.lv/2,
F, F, T, T, "c") # Aprx
```

```
normal.and.t.dist(NA, NA, ptilde.diff, sdtilde, n0tilde, NA, aprx.lv/2,
aprx.lv/2, F, F, T, T, "c") # Aprx

# EXERCISES FOR § 5-6
```

§ 5-7 Summary Tables for Inference Procedures for Two Samples

Refer to § 4-9.

§ 5-8 What if We Have More than Two Samples?

§ 5-8.1 Completely Randomized Experiment and Analysis of Variance

Table 5-5 Tensile Strength of Paper (psi)

Figure 5-7 (a) Box plots of hardwood concentration data. (b) Display of
the model in equation 5-31 for the completely randomized single-factor
experiment.

Table 5-6 Typical Data for a Single-Factor Experiment

Table 5-7 The Analysis of Variance for a Single-Factor Experiment

EXAMPLE 5-14 Tensile Strength

Table 5-8 Minitab Analysis of Variance Output for the Paper Tensile
Strength Experiment

Figure 5-8 Tensile strength averages from the hardwood concentration
experiment in relation to a normal distribution with standard deviation
sqrt(MS_E/n) = sqrt(6.51/6) = 1.04.

Table 5-9 Residuals for the Tensile Strength Experiment

Normal Probability Plot Interpretation

Figure 5-9 Normal probability plot of residuals from the hardwood
concentration experiment.

Figure 5-10 Plot of residuals versus factor levels (hardwood
concentration).

Figure 5-11 Plot of residuals versus \bar{y}_i.

§ 5-8.2 Randomized Complete Block Experiment

Figure 5-12 A randomized complete block design.

Table 5-10 A Randomized Complete Block Design

Table 5-11 A Randomized Complete Block Design with a Treatments and b
Blocks

Table 5-12 Analysis of Variance for a Randomized Complete Block Design

EXAMPLE 5-15 Fabric Strength

Table 5-13 Fabric Strength Data—Randomized Complete Block Design

Table 5-14 Analysis of Variance for the Randomized Complete Block
Experiment

Table 5-15 Minitab Analysis of Variance for the Randomized Complete Block
Design in Example 5-15

Figure 5-13 Strength averages from the fabric experiment in relation to a
normal distribution with standard deviation sqrt(MS_E/b) = sqrt(0.0792/5)
= 0.126.

Table 5-16 Residuals from the Randomized Complete Block Design

Figure 5-14 Normal probability plot of residuals from the randomized
complete block design.

Figure 5-15 Residuals versus y_{ij}^.

Figure 5-16 Residuals by chemical type.

Figure 5-17 Residuals by block.

EXERCISES FOR § 5-8